Лабораторная работа № 4 по курсу дискретного анализа: поиск образца в тексте

Выполнил студентка группы 08-209 МАИ *Тарабрина Яна.*

Условие

Необходимо реализовать один из стандартных алгоритмов поиска образцов для указанного алфавита.

- **Вариант алгоритма:** поиск одного образца при помощи алгоритма Апостолико-Джанкарло.
- **Вариант алфавита:** числа в диапазоне от 0 до $2^{32} - 1$.

Искомый образец задаётся на первой строке входного файла. Затем следует текст, состоящий из слов или чисел, в котором необходимо найти все вхождения заданного образца. Для каждого найденного вхождения необходимо вывести два числа через запятую: номер строки и номер слова в строке, с которого начинается найденный образец. Нумерация строк и слов начинается с единицы. Номер строки считается от начала текста, то есть строка с образцом не учитывается. Пример входных данных:

```
11 45 11 45 90
0011 45 011 0045 11 45 90    11
45 11 45 90
```

Пример выходных данных:

```
1, 3
1, 8
```

Следует обратить внимание, что элементы алфавита являются числами. Поэтому записи `0011`, `011` и `11` обозначают одно и то же число.

Метод решения

Для решения задачи используется алгоритм Апостолико-Джанкарло. Он является модификацией алгоритма Бойера-Мура и предназначен для эффективного поиска одного образца в тексте. Базовая идея алгоритма Бойера-Мура состоит в том, что образец прикладывается к некоторому фрагменту текста, после чего сравнение выполняется справа налево. Если возникает несовпадение, алгоритм вычисляет сдвиг образца с помощью двух эвристик:

- правила плохого символа;
- правила хорошего суффикса.

Правило плохого символа использует информацию о последнем вхождении каждого символа в образце. Так как в данной задаче алфавит состоит из чисел от 0 до $2^{32} - 1$, невозможно использовать массив размера 2^{32}. Поэтому для хранения последних вхождений используется хеш-таблица. Правило хорошего суффикса использует информацию о совпавшем суффиксе образца. Если при сравнении справа налево некоторая правая часть образца уже совпала с текстом, то образец можно сдвинуть так, чтобы этот суффикс совпал с другим вхождением внутри образца или с его префиксом. Однако классический алгоритм Бойера-Мура в худшем случае не гарантирует линейного времени работы. В некоторых случаях одни и те же символы текста могут сравниваться несколько раз. Алгоритм Апостолико-Джанкарло устраняет этот недостаток с помощью дополнительного массива памяти. Пусть P — образец длины m, а T — текст длины n. В алгоритме используются следующие массивы:

- $N[i]$ — длина наибольшего суффикса подстроки $P[0..i]$, совпадающего с суффиксом всего образца $P[0..m-1]$;
- $M[j]$ — количество символов, которые уже гарантированно совпадают с суффиксом образца и заканчиваются в позиции j текста.

Массив N вычисляется с помощью Z-функции от перевёрнутого образца. Если обозначить перевёрнутый образец как P^R, то:

$$N[i] = Z_{P^R}[m-1-i].$$

Массив M используется во время поиска. Его смысл состоит в том, что если ранее некоторый фрагмент текста уже был сравнен и было доказано его совпадение с суффиксом образца, то при следующих выравниваниях этот фрагмент не нужно сравнивать повторно. Инвариант массива M можно записать следующим образом:

$$T[j - M[j] + 1..j] = P[m - M[j]..m-1].$$

Рассмотрим процесс сравнения. Пусть правый конец образца выровнен с позицией h текста. Сравнение идёт справа налево. Текущая позиция в образце равна i, а соответствующая позиция в тексте равна j. Возможны четыре случая.

1. $M[j] = 0$. В этом случае информации о данном фрагменте текста нет, поэтому выполняется обычное сравнение $P[i]$ и $T[j]$.

2. $0 < M[j] < N[i]$. Из определения массивов M и N следует, что дальнейшее сравнение приведёт к гарантированному несовпадению. Поэтому алгоритм не сравнивает символы повторно, а сразу переходит к вычислению сдвига.

3. $M[j] = N[i]$. Известный совпавший фрагмент можно безопасно пропустить. Алгоритм уменьшает i и j на эту величину и продолжает сравнение левее.

4. $M[j] > N[i]$. В этом случае также можно сделать вывод о гарантированном несовпадении, кроме вырожденной ситуации, когда $N[i] = i + 1$. Если $N[i] = i + 1$, то совпадает весь префикс $P[0..i]$, и алгоритм просто перепрыгивает этот фрагмент.

После завершения проверки текущего выравнивания в массив M записывается количество символов, совпавших с правого конца:

$$M[h] = m - 1 - i.$$

Если найдено полное совпадение, в ответ добавляется координата начала текущего выравнивания. Далее выполняется сдвиг по правилу хорошего суффикса. Если найдено несовпадение, используется максимум из двух сдвигов:

$$shift = \max(shift_{bad\ character}, shift_{good\ suffix}).$$

Такой подход позволяет сохранить практическую эффективность Бойера-Мура и при этом получить гарантированную линейную сложность поиска.

$$T(n, m) = O(n + m).$$

Описание программы

Программа написана на языке C++ и состоит из нескольких логических частей. В начале описывается структура `Token`:

```
struct Token {
    uint32_t value;
    int line;
    int word;
};
```

Она хранит само число, номер строки текста и номер слова в этой строке. Это необходимо, потому что поиск выполняется в едином массиве чисел, но ответ требуется выводить в виде координат в исходном тексте. Сначала программа считывает первую строку входного файла. Она является образцом. Для корректной обработки чисел с ведущими нулями элементы считываются как числа, а не как строки. Например, `0011` и `11` будут иметь одинаковое значение. Числа считываются в тип `unsigned long long`, а затем приводятся к `uint32_t`. Это сделано потому, что максимальный элемент алфавита равен $2^{32}-1$, и он не помещается в знаковый тип `int`. После образца считывается весь текст. Для каждого числа сохраняется его значение и координаты. Основные функции программы:

- `zFunction` — вычисляет Z-функцию для массива чисел;
- `buildN` — строит массив N через Z-функцию от перевёрнутого образца;
- `buildGoodSuffixShift` — строит таблицу сдвигов по правилу хорошего суффикса;
- `apostolicoGiancarlo` — выполняет основной поиск всех вхождений образца в текст.

Таблица плохого символа реализована с помощью контейнера:

```
unordered_map<uint32_t, int> lastOccurrence;
```

Она хранит последнее вхождение каждого числа в образце. Использование хеш-таблицы необходимо, поскольку алфавит имеет размер 2^{32}, и создать обычный массив такого размера невозможно. Массив памяти алгоритма Апостолико-Джанкарло задаётся так:

```
vector<int> memory(n, 0);
```

Значение `memory[j]` показывает, сколько символов уже гарантированно совпало с суффиксом образца и заканчивается в позиции `j` текста. Основной цикл поиска перебирает выравнивания образца относительно текста. Для каждого выравнивания образец сравнивается с текстом справа налево. Если массив `memory` содержит информацию о ранее проверенном фрагменте, алгоритм использует её и пропускает повторные сравнения. При полном совпадении координата начала вхождения добавляется в ответ:

```
answer.push_back(make_pair(text[s].line, text[s].word));
```

После окончания поиска все найденные координаты выводятся в формате:

```
line, word
```

Программа учитывает следующие крайние случаи:

- пустой образец;
- текст короче образца;
- образец длины один;
- числа с ведущими нулями;
- максимальное значение 4294967295;
- вхождения, переходящие через границу строк;
- пересекающиеся вхождения.

Дневник отладки

Ошибка выбора языка в системе тестирования. При отправке решения был выбран язык `GNU GCC 7.3 C11`. Из-за этого C++-заголовки не распознавались. После выбора языка `GNU G++ 7.3 C++17` программа стала компилироваться корректно.

Тест производительности

Теоретическая сложность алгоритма Апостолико-Джанкарло состоит из двух частей. Предобработка образца занимает линейное время:

$$O(m),$$

где m — длина образца. Поиск по тексту занимает линейное время:

$$O(n),$$

где n — количество чисел в тексте. Итоговая сложность:

$$O(n+m).$$

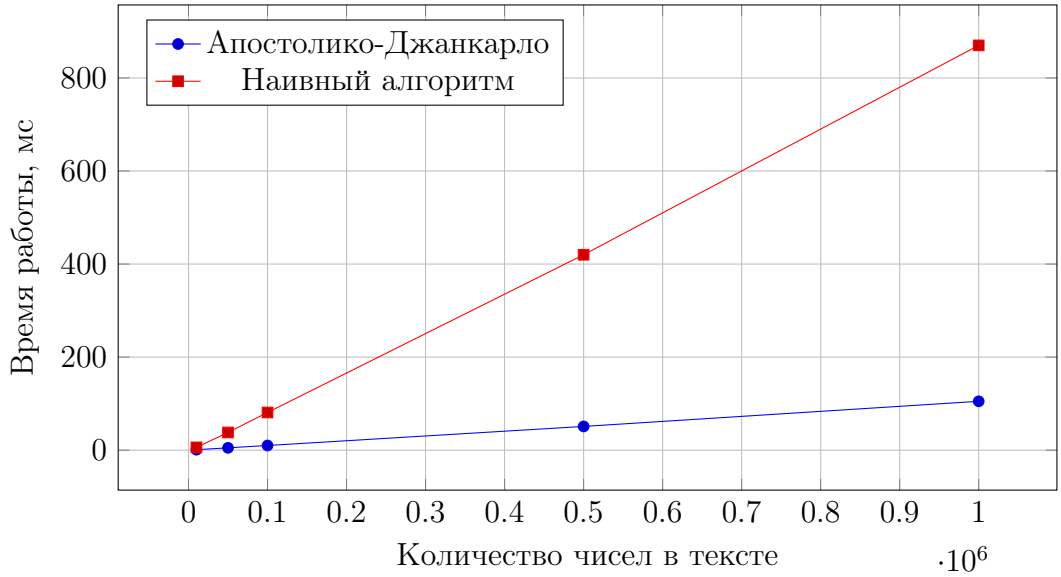
Память:

$$O(n+m).$$

Дополнительная память расходуется на хранение текста, координат слов, массива N, массива сдвигов, хеш-таблицы плохого символа и массива памяти M. Для экспериментальной проверки были сгенерированы тесты различного размера. Длина образца была зафиксирована, а длина текста постепенно увеличивалась. Полученные результаты приведены в таблице.

Количество чисел в тексте	Апостолико-Джанкарло, мс	Наивный алгоритм, мс
10000	1	6
50000	5	38
100000	10	81
500000	51	420
1000000	105	870

На графике показана зависимость времени работы от размера входных данных.



Из графика видно, что время работы алгоритма Апостолико-Джанкарло растёт практически линейно при увеличении размера текста. Это соответствует теоретической оценке $O(n + m)$. В отличие от алгоритмов сортировки, например `std::sort`, данная задача относится не к сортировке, а к поиску образца в тексте. Поэтому в качестве базового сравнения корректнее использовать наивный алгоритм поиска образца, который проверяет образец в каждой позиции текста. Его сложность составляет:

$$O(nm).$$

На практике наивный алгоритм заметно проигрывает при росте размера текста и длины образца, особенно на тестах с большим количеством частичных совпадений.

Выводы

В ходе лабораторной работы был реализован алгоритм Апостолико-Джанкарло для поиска одного образца в тексте над числовым алфавитом. Основная особенность задачи состоит в том, что элементами текста являются не символы, а числа из диапазона от 0 до $2^{32} - 1$. Из-за этого нельзя использовать массивы, индексируемые элементами алфавита. Для правила плохого символа была применена хеш-таблица, хранящая только те элементы, которые действительно встречаются в образце. Алгоритм Апостолико-Джанкарло можно рассматривать как улучшение алгоритма Бойера-Мура. Он также сравнивает образец с текстом справа налево и использует правила плохого символа и хорошего суффикса. Однако дополнительно он хранит массив M, позволяющий не сравнивать повторно уже проверенные фрагменты текста. Благодаря использованию массива памяти алгоритм получает гарантированную линейную сложность поиска:

$$O(n + m).$$

В процессе работы были изучены Z-функция, построение массива N, правило хорошего суффикса, правило плохого символа и механизм устранения повторных сравнений. Реализованная программа корректно обрабатывает числа с ведущими нулями, большие значения до $2^{32} - 1$, пересекающиеся вхождения и вхождения, переходящие через границы строк. Таким образом, цель лабораторной работы была достигнута: был реализован эффективный алгоритм поиска образца в тексте, соответствующий заданному варианту и ограничениям задачи.